CS 3600 Intro to Artificial Intelligence
Homework 4
Bayes' Rule

Aaron Hillegass

This homework is due at 11:59 pm on Monday, Sept 22. It must be uploaded to Gradescope by then. No credit will be given for late submissions.

1 Part 1: Dice

Your friend has two pairs of dice. She rolls the first pair and notes their sum (But does not tell you what that sum is!) She repeatedly rolls the second pair and tells you if the new sum is higher than (H), lower than(L), or equal to (E) the first pair. You are trying to guess what the sum of the first pair of dice is.

Based on the squence of H, L, and E, you need to guess what the original sum was. Of course, you can't really be sure. So you will return a list of probabilities for each possible original sum.

For example, if she tells you HHL, you would compute the following probabilities:

Sum	Probability
2	0
3	0.025953
4	0.096518
5	0.193322
6	0.262744
7	0.241295
8	0.125116
9	0.044613
10	0.009652
11	0.000786
12	0

So your best guess for this round is 6, but there is only about a 26% chance that you are right.

1.1 Your Task

The problem files starts with the number of dice, the number of sides on each die, and the number of rounds. Then there is a round of the game on each line. (For each round, she re-rolls the dice.)

```
2 6 12
H
L
E
HL
HHL
EEEEEE
HHHHHHHH
LLLLLLLL
HHHHHHHHHHHH
LLLLLLLLLLLL
HHHHHHHHHHHHE
LLLLLLLLLLLLE
```

The `dice-main.py` will read this file. It will initiate your `Belief` class with the number of dice and the number of sides on each die. Then it will call `analyze` for each round.

Your job, then, is to write code that looks at the assumptions (number of dice and the number of sides on each die) and the evidence (the sequence of letters) and returns what you now believe about the original roll.

When it runs on this example file above, it will look like this:

```
> python3 dice-main.py problems/dice-example-small.txt
Belief analyzing problem with 2 dice, each with 6 sides.
       P(d|H)| 2:0.061 3:0.115 4:0.157 5:0.181 6:0.183 7:0.157 8:0.087 9:0.042 10:0.016 11:0.003 12:0.000
       P(d|L)| 2:0.000 3:0.003 4:0.016 5:0.042 6:0.087 7:0.157 8:0.183 9:0.181 10:0.157 11:0.115 12:0.061
       P(d|E)| 2:0.007 3:0.027 4:0.062 5:0.110 6:0.171 7:0.247 8:0.171 9:0.110 10:0.062 11:0.027 12:0.007
      P(d|HL)| 2:0.000 3:0.012 4:0.050 5:0.116 6:0.196 7:0.251 8:0.196 9:0.116 10:0.050 11:0.012 12:0.000
     P(d|HHL)| 2:0.000 3:0.026 4:0.097 5:0.193 6:0.263 7:0.241 8:0.125 9:0.045 10:0.010 11:0.001 12:0.000
  P(d|EEEEEE)| 2:0.000 3:0.000 4:0.005 5:0.035 6:0.165 7:0.591 8:0.165 9:0.035 10:0.005 11:0.000 12:0.000
P(d|HHHHHHHH)| 2:0.279 3:0.348 4:0.244 5:0.103 6:0.023 7:0.002 8:0.000 9:0.000 10:0.000 11:0.000 12:0.000
P(d|LLLLLLLL)| 2:0.000 3:0.000 4:0.000 5:0.000 6:0.000 7:0.002 8:0.023 9:0.103 10:0.244 11:0.348 12:0.279
P(d|HHHHH...)| 2:0.387 3:0.382 4:0.183 5:0.044 6:0.004 7:0.000 8:0.000 9:0.000 10:0.000 11:0.000 12:0.000
P(d|LLLLL...)| 2:0.000 3:0.000 4:0.000 5:0.000 6:0.000 7:0.000 8:0.004 9:0.044 10:0.183 11:0.382 12:0.387
P(d|HHHHH...)| 2:0.204 3:0.403 4:0.289 5:0.092 6:0.011 7:0.000 8:0.000 9:0.000 10:0.000 11:0.000 12:0.000
P(d|LLLLL...)| 2:0.000 3:0.000 4:0.000 5:0.000 6:0.000 7:0.000 8:0.011 9:0.092 10:0.289 11:0.403 12:0.204
PASSED: Matched solution in problems/sol-example-small.txt
```

1.2 Bayes' Rule

You will need to use Bayes' Rule, which says: The probability that the original sum was d given a sequence s is given by

$$P(d|s) = \frac{P(s|d)P(d)}{P(s)}$$

To calculate $P(s)$, you will use:

$$P(s) = \sum_{d \in D} P(s|d)P(d)$$

1.3 Dealing with all possible combinations of dice

How do you figure out how many combinations of 5 seven-sided dice sum to 18?

Here's a hint: If you write the following code

```
import itertools

for r in itertools.product([1,2,3,5], repeat=3):
    print(r)
```

The result will be:

```
(1, 1, 1)
(1, 1, 2)
(1, 1, 3)
(1, 1, 5)
(1, 2, 1)
(1, 2, 2)
...
(5, 5, 3)
(5, 5, 5)
```

1.4 Belief.py

In the initializer method, you will be given the number of dice and the number of sides on each die. You should use this opportunity to construct lookup tables that will speed up your `analyze` method.

In the `analyze` method, you will be given a list of characters that represent a round of play. For example, you might receive `[H,L,L,H]`. You will return a numpy array containing your beliefs about the sum of the original roll.

1.5 numpy

You will be doing all your calculations using numpy arrays. numpy is a very popular library for working with vectors, matrices, and tensors. If you haven't used it before, you get to learn it now.

One of the things you should know: For performance reasons, you can't change the size of a numpy array after it is created.

Python is a pig of a language: a program written in Python will take about 70 times as long and consume about 70 times as much electricity as a similar program written in C/Rust/C++. (`https://arxiv.org/html/2410.05460v1`) The numpy library is used from Python, but it is written in C++. The trick to getting acceptable performance out of your Python program is to let the numpy library do the heavy lifting.

Most engineers using numpy for the first time use *way* too many loops. Then they learn the functions of the library better and how to use broadcasting. I'm going to help you with this process: **You are not allowed to use more than 4 loops in `Belief.py` or `LongBelief.py`. There is already one in `show_array`. So you get to use three in each file. (List comprehensions count as loops.)**

1.6 Working in Log Space

If your `Belief` class is used for very long sequences of rolls, it will underflow the floating-point numbers on your computer. Try it:

```
> python3 dice-main.py problems/dice-01-big.txt
```

After a few rows, you should get a bunch `NaN` values when you underflow the floating-point numbers.

To get around this, you can use logarithms to deal with very small numbers.

You will do this in `LongBelief.py`. It should have the exact same functionality as `Belief.py`, but it will be able to deal with very long sequences of rolls.

In `LongBelief.py`, your lookup tables will contain the log of the probabilities. You will add those instead of multiplying them. At the end, just before dividing by the marginal likelihood, you will exponentiate them (e^x) so that you get exactly the same results as the previous exercise. Sadly, if x is really tiny, you will not get a very accurate value for e^x.

Fortunately, in this case x is being used with several other numbers. Let's say x, y, and z are all in "log space". And you really want to exponentiate them and compute a ratio r:

$$r = \frac{e^x}{e^y + e^z}$$

In scientific computing, is common to add a constant c to x, y, and z (so they aren't such huge negative numbers) before exponentiating. This does not change the value of r:

$$r = \frac{e^{x+c}}{e^{y+c} + e^{z+c}} = \frac{e^x e^c}{e^y e^c + e^z e^c} = \frac{e^x}{e^y + e^z}$$

It can, however, make the calculation more accurate by moving the exponents into a reasonable scale for doing floating point operations.

What is a good value for c? Most people use

$$c = -\max(x, y, z)$$

So the largest exponent becomes 0.

Also: The log of zero is not defined. Use `-np.inf`. Here's a way to do this in code:

```
log_p = np.log(p, where=p > 0)
log_p[p == 0] = -np.inf
```

To use the `LongBelief` class, append the word "long" to the command line:

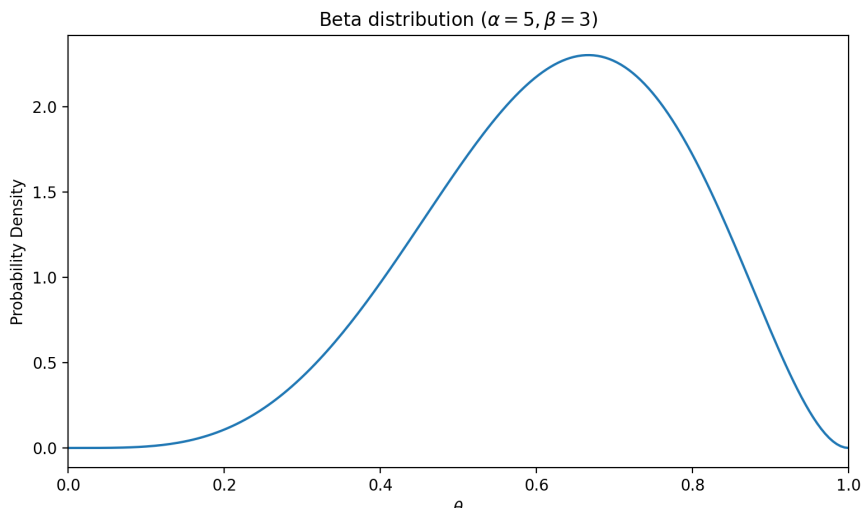
```
> python3 dice-main.py problems/dice-01-big.txt long
LongBelief analyzing problem with 3 dice, each with 7 sides.
P(d|LLLLL...)| 3:0.000 4:0.000 5:0.000 6:0.000 7:0.000 8:0.000 ...
...
PASSED: Matched solution in problems/sol-01-big.
```

`LongBelief.py` should give correct results for the short sequences too! Please double-check this before you turn them in.

2 Part 2: Coins

In the first part, you used Bayes' Rule to update your beliefs about a discrete random variable. In this part, you will do the same thing for a continuous random variable.

Your friend Rick makes crooked coins. The chance of the coin landing with heads up is some number θ for each coin. His factory makes an assortment of coins – the value of θ coming out of his factory follows a beta distribution ($\alpha = 5, \beta = 3$).



Each coin was neatly labeled with its θ value, but his basement flooded and all the labels were washed away.

Rick has done a series of experiments on each coin. Each coin is represented by a file. Each line of the file represents an experiment performed on that coin. Here is an example line:

```
4 9
```

This line represents "I decided to see how many flips it took to get a total of 4 heads. When I actually flipped this coin, I got that fourth head on the ninth flip."

Before all the experiments, on the first line of the file is a floating point number less than 1:

```
.70
```

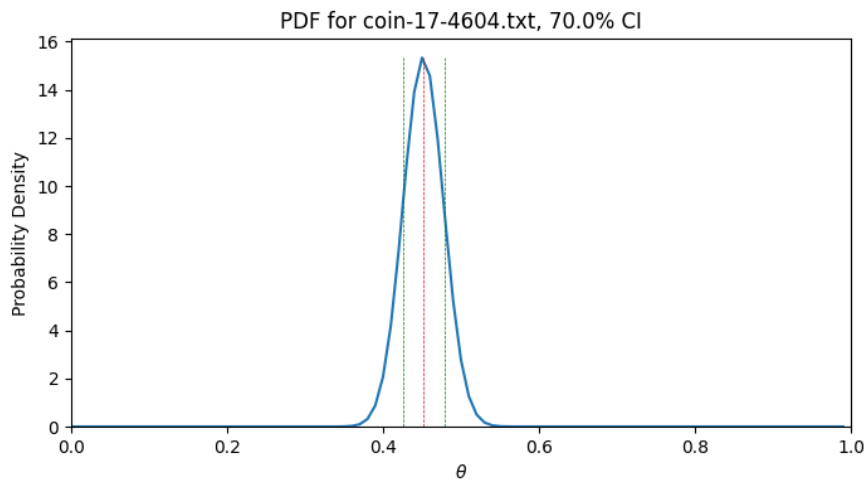
This is, "The buyer needs to know the smallest interval that we can be 70% confident that θ is in.

When your program runs, it will look like this:

```
> python3 coin-main.py problems/coin-17-4604.txt
Loaded 17 experiments.
```

```
Maximum a posteriori estimation of theta: 0.4520
70.0% credibility interval: 0.4250 - 0.4790
CORRECT
```

It will also create a plot of your beliefs about the value of θ for the coin:



2.1 The Negative Binomial Probability Distribution

Given the probability of success θ and a goal number of successes r, the probability of having k failures before seeing r successes is given by:

$$P(k) = \binom{k+r-1}{k}(1-\theta)^k\theta^r$$

Notice that this is different from the numbers you are being given; you are being given r and $r+k$.

You can learn more about the negative binomial distribution here: `https://en.wikipedia.org/wiki/Negative_binomial_distribution`

2.2 CoinEstimator

You are creating an agent in `CoinEstimator.py`:

- In `prepare_to_estimate()`, it will be given a list of all the experiments as a list of tuples: `(r, w)`.

  Create a numpy array containing the approximate the probability density function for θ based on the prior and the likelihood of the outcomes of these experiments for 1000 points between 0 and 1. Note: This should integrate to 1.0.

From the SciPy framework, you can use `beta` to find the log-prior and `nbinom` to find the log-likelihoods. (In order to work with a large number experiments, you should do the math in log-space and convert it back to non-log-space when you create the final estimate of the PDF.)

You may use exactly one loop in this method.

- `max_a_posteriori` method will return the value of θ that has the highest probability density. It is already implemented for you.
- `probability_density` method takes as an argument a possible value of θ and returns the probability density for it. It is already implemented for you.
- Your `credibility_interval` method takes an argument (like 0.7) and returns the shortest interval that contains that proportion (70%) of the area under the curve.

  You may use exactly one loop in this method.

3 Test Your Code

To make sure your solutions work correctly, you should run the provided test cases and also create your own.

3.1 Running the Dice Problems

To test your `Belief.py` on a sample problem file (for example, `problems/dice-example-small.txt`), run:

```
> python3 dice-main.py problems/dice-example-small.txt
```

To test your `LongBelief.py` (which works in log space), add the word `long` to the command line:

```
> python3 dice-main.py problems/dice-01-big.txt long
```

3.2 Creating Dice Test Cases

You can generate your own dice problems using `make_dice_problem.py`:

```
Usage:
  python3 make_dice_problem.py <num_dice> <num_sides> <rounds> <minlen> <maxlen>
```

Example:

```
> python3 make_dice_problem.py 2 6 12 2 8
```

This creates `prob-2-8.txt`. The first line has the number of dice, the number of sides, and the number of rounds (`2 6 12`). Each following line is a sequence of `H`/`L`/`E` of length between `minlen` and `maxlen`.

3.3 Running the Coin Problems

To test your `CoinEstimator.py` on a coin problem file (for example, `problems/coin-17-4604.txt`), run:

```
> python3 coin-main.py problems/coin-17-4604.txt
```

3.4 Creating Coin Test Cases

You can generate your own coin problems using `make_coin_problem.py`:

```
Usage:
  python3 make_coin_problem.py <number of experiments> <outfilename>

Example:
> python3 make_coin_problem.py 20 problems/my-coin-test.txt
```

This creates `problems/my-coin-test.txt` with 20 experiments. The first line is a credibility level (randomly chosen in $[0.50, 0.90]$). Each following line has two integers: the target number of heads (`r`) and the trial on which the rth head appeared.

4 Criteria for success

You will submit your `Belief.py`, `LongBelief.py`, and `CoinEstimator.py` to Gradescope. They will be tested on problems that you have not seen.

Do not import any additional libraries.

The autograder will check to see that you have not used excessive loops. `Belief.py` and `LongBelief.py` must have no more than four loops. `CoinEstimator.py` must have no more than two loops.

Do this work by yourself. A hint from another student is OK. Looking at another student's code is *not* OK. Any use of an LLM is forbidden.